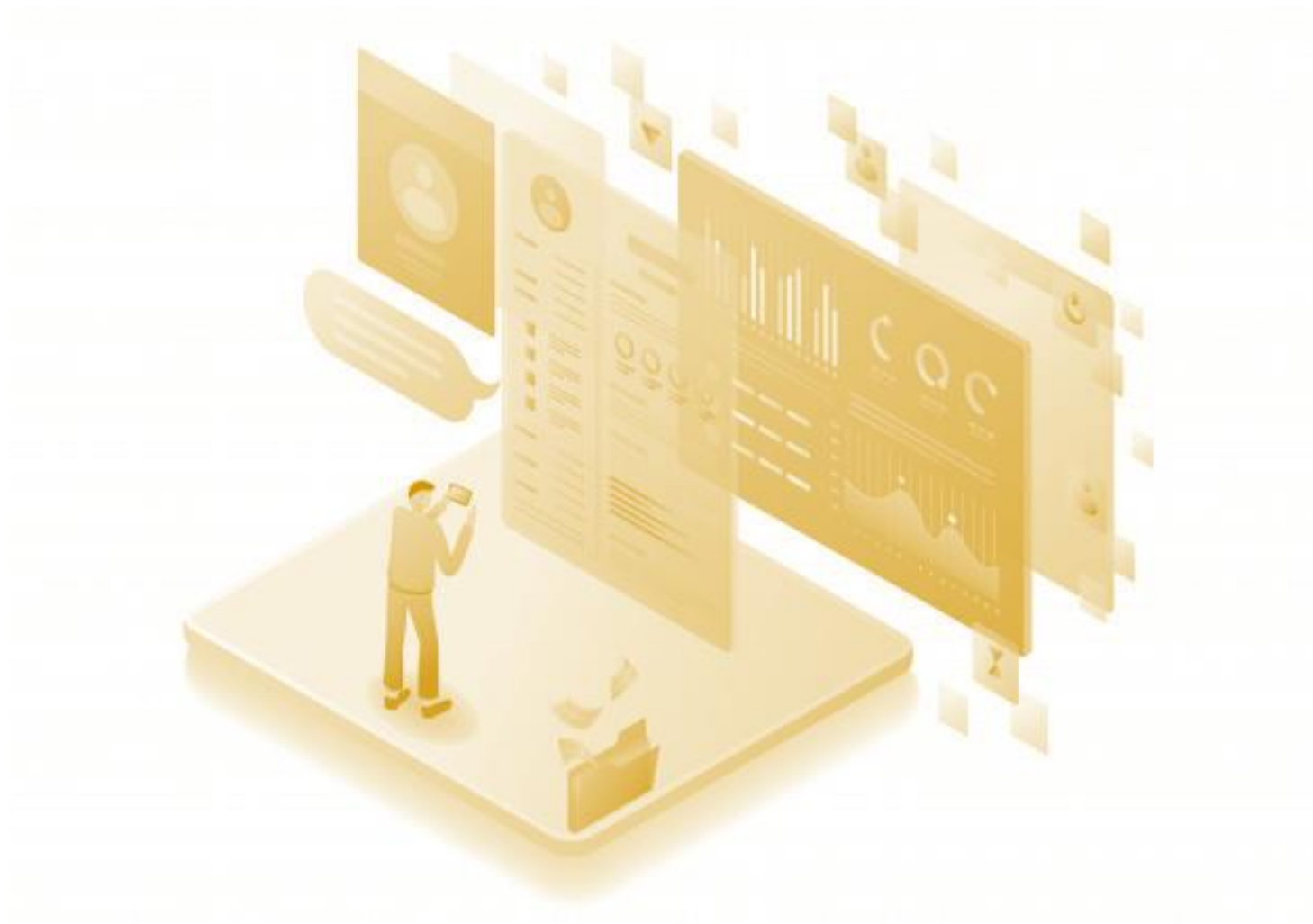




GUI tervezés



ÓBUDAI EGYETEM
NEUMANN JÁNOS INFORMATIKAI KAR

MODUL 1

Grafikus felület API-k

User32.dll MessageBox példa

Platform independent API-k

WPF Hello World

Vezérlők

Események

- **GUI:** Graphical User Interface
- Olyan alkalmazások írása C# nyelven, ahol gombokat, listákat, képeket, egyéb vezérlőket tudunk adatbevitelre és megjelenítésre használni
- Keretrendszerünk: WPF (Windows Presentation Foundation)
- Nem alkotunk GUI-t nulláról programozva!
 - WPF: WINAPI-n keresztül User32.dll és Comctl32.dll-ek metódusainak hívogatása
 - Platformfüggő C++ nyelven írt osztályok és metódusok (UI mindig az..)
 - WPF számunkra ezt C# osztályokon keresztül teszi elérhetővé

- **Régi (nem javasolt)**

- Carbon (Mac OS X, dead)
- Cocoa/ObjectiveC (Mac OS X, deprecated?)
- Java AWT/Swing (Java, not developed)
- MFC (Windows, C++, dead)
- Windows Forms (Windows XP, .NET, ??not developed??)
- Metro (Windows RT) (Windows 8, C#, C++, dead)
- Universal apps (UWP) (Windows 10, C#, dead → WinUI)

- **Újabb (jelenleg elérhető és javasolt)**

- Cocoa/Swift (Mac OS X)
- Java FX/TornadoFX (Java/Kotlin)
- VCL / FireUI (Windows/Pascal/C++, deprecated?)
- WxWidgets/FLTK (Linux/Unix, C++)
- QT (Linux/Unix/Windows, C++)
- GTK/GTK+ (Linux/Unix/Windows, C++)
- WPF (Windows XP/7, C# ??? haldoklik ???)

- .NET 2.0 (2002) óta elérhető
- Csak natív WINAPI elemekhez menedzselt hozzáférés
- MFC rendszer C# nyelvű átírata
- Van platformfüggetlen változata is → Mono
- **Előnyei**
 - Kiforrott, letisztult, stabil
 - Egyszerű, klasszikus felületek
 - Ablakok, dialógusok, sok vezérlő
- **Hátrányai**
 - Csak közvetlen események
 - Csak név alapú elérés
 - Felület C# nyelven írható le (nincs jó UI-Logic szétválasztás)
 - Reszponzivitás teljes hiánya

- Windows Metro (UWP): hibás stratégia, elrontott projekt
- **WPF**
 - Gazdag grafika, jó multimédiás támogatottság
 - Nagyon egyénre szabható
 - Adatkötések, továbbított események
 - Platformfüggő → csak Windows
- **Platformfüggetlen UI**
 - .NET Core esetében mindenki ezt várta → nem lesz
 - Helyette: MAUI (Multiplatform APP UI) → már .NET 6-ban preview-ként benne van

- **XAML, C#**
 - Telerik UI
 - Avalonia UI
 - Xamarin.Forms (2016: Microsoft, 2020: átnevezve MAUI-ra)
 - ETO.Forms
 - Uno.Platform
- **Mobile cross-platform (Web/JS-centered???)**
 - Flutter (DEC/2018)
 - Cordova, Phonegap
 - Ionic (Angular)
 - ElectronUI
 - React Native
- **Web-centered**
 - WebAssembly
 - C# Blazor (🔗 !!!)

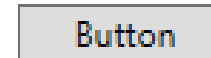
- Nem biztos, hogy 2-3 éve múlva még fogják használni
 - Desktop appok mennek ki a divatból...
- De nagyon sok része más technológiákban előkerül
 - XAML nyelvű leírás → Xamarin, MAUI ugyanez (UI vezérlők neve más...)
 - Üzleti logika leválasztás (MVVM+Adatkötés) → Xamarin, MAUI ugyanez de JS rendszerek is hasonlóak
 - Iparban használják sok helyen → itt megmarad
- Tehát a WPF tudás hasznos, innen csak 1 lépés:
 - Mobilfejlesztés pl. Xamarinnal
 - Kliens oldali webfejlesztés pl. Angularral

- Írjunk egy Hello World alkalmazást WPF-ben!

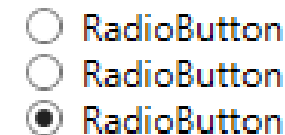


- Kategóriák
 - **ContentControl**
 - Content-ben valamilyen object → akár másik vezélő
 - **ItemsControl**
 - Items-ben több object
 - **Panel (tartalommenedzser)**
 - Több UI elemet fognak össze
 - **Egyebek**
 - Egyszerűbb vezérlők

- **Button** (nyomógomb)
 - **Content** tulajdonság tartalmazza a szöveget (vagy más)
 - **Click** esemény
- **Label** (szöveg)
 - **Content** tulajdonság tartalmazza a szöveget (vagy más)
- **CheckBox** (jelölőnégyzet)
 - **Content** tulajdonság tartalmazza a szöveget (vagy más)
 - **IsChecked** tulajdonság tartalmazza, hogy be van-e jelölve
 - **Checked/Unchecked** események
- **RadioButton** (rádiógomb)
 - **Content** tulajdonság tartalmazza a szöveget (vagy más)
 - **GroupName** csoportosítja, de nélkül is biztosít a csoportosításra logikát
 - **IsChecked** tulajdonság
 - **Checked/Unchecked** események



Label



- **TextBlock** (Label alternatíva)
 - **Text** tulajdonságban van a szöveg
- **TextBox** (szövegbeviteli mező)
 - **Text** tulajdonságban van a szöveg
 - **TextChanged** esemény
- **ListBox** (lista) és **ComboBox** (legördülő lista)
 - **Items** tulajdonság tartalmazza az elemeket
 - **SelectionChanged** esemény
 - **SelectedItem** tulajdonság → ref. Kijelölt elem

TextBlock

TextBox

Item1
Item2
Item3

Item1
Item2
Item3

- **Foreground/Background**: írószín és háttérszín
- **x:Name** vagy **Name**: ilyen néven hivatkozhatunk a vezérlőre
- **Margin/Padding**: külső és belső margó
- **BorderBrush/BorderThickness**: keretszín és keretvastagság
- **FontFamily/FontSize/FontWeight/FontStyle/FontStretch**: betűtípus
- **Opacity**: átlátszóság
- **Visibility**: láthatóság
- **IsEnabled**: bevitel tiltás
- **Tag**: 1 db tetszőleges objektum elhelyezése a vezérlőben
- **HorizontalContentAlignment/VerticalContentAlignment**: saját tartalom pozícionálása
- **HorizontalAlignment/VerticalAlignment**: ő maga a tartalommenedzserén belül hova rendeződjön
- **Width/Height**: szélesség/magasság

- Írjunk egy kamatos kamat számító programot!
- Lehesen megadni
 - Kívánt összeget
 - Kamatlábat
- Egy listboxban jelenítsük meg, hogy mekkora lenne a tartozásunk
 - 1...10...20 év stb. esetén (50 évet generáljunk le)
- Az adott listbox elemet kijelölve jelenjen meg a havi törlesztőrészlet!



- Céljuk az elemek meghatározott szabályok szerinti elrendezése és átméretezés kezelése
 - saját grafikus megjelenésük nincs!
 - Mindegyiknek Children tulajdonsága tartalmazza az elemeket
- **Grid**
 - elemek rácsszerű elrendezése, automata átméretezés
- **Canvas**
 - elemek adott pixelre való elhelyezése
- **DockPanel**
 - jobbra/balra/fel/le/középre igazított elhelyezés
- **StackPanel**
 - elemek egymás utáni elhelyezése
- **WrapPanel**
 - elemek egymás utáni elhelyezése sortöréssel

- Pixelpontos elhelyezést tesz lehetővé
- Reszponzivitás nincs
- Minden tartalmazott elem örököl egy Left és Top jellemzőt (+ Right, Bottom)
- Egy elem pozícionálása
 - Left
 - Top
 - Width
 - Height

- Jobbra/balra/le/fel igazítása az elemeknek
- Minimum 2db vezérlő esetén van hatása
- Minden tartalmazott elem örököl egy Dock jellemzőt
- Egy elem pozícionálása
 - **Dock** : Left | Right | Top | Bottom

- Elemek egymás után helyezése vízszintesen vagy függőlegesen
 - **Orientation:** Horizontal | Vertical
- Nincs örökölt jellemző
- Ha elfogy a hely:
 - **StackPanel:** scrollbar
 - **WrapPanel:** sor/oszloptörés

- Még elemek behelyezése előtt egy nxn-es rácsszerkezet definiálható
 - * jelentése: arányszám
- Minden elem örököl egy Row és Column tulajdonságot
- Ezzel megadható, hogy melyik cellába kerüljön
- Ott kitölti a rendelkezésre álló területet

```
<Grid>  
  <Grid.ColumnDefinitions>  
    <ColumnDefinition Width="1*" />  
    <ColumnDefinition Width="3*" />  
    <ColumnDefinition Width="1*" />  
  </Grid.ColumnDefinitions>  
  <Grid.RowDefinitions>  
    <RowDefinition Height="1*" />  
    <RowDefinition Height="1*" />  
    <RowDefinition Height="1*" />  
  </Grid.RowDefinitions>  
</Grid>
```

- Írjunk egy Pizzarendelő alkalmazást!
 - Egy űrlapon lehessen megadni a pizzák
 - **Névét** (legördülő menüből)
 - **Alapját** (radiobutton)
 - **Extrákat** (checkbox)
 - **Megrendelő nevét** (textbox)
 - Egy listboxban lássuk a megrendelt pizzákat!
 - Az alkalmazás bezárásakor mentődjenek el a pizzák .json fájlba!
 - Újraindításkor töltődjenek vissza!



Köszönöm a figyelmet!

Kérdés esetén e-mailben szívesen állok rendelkezésre.